

Project Name: AI Dungeon Story Generator

APP URL

<https://aiml-intern-project-3-ai-dungeon-story-generator-p6ydiума92pew.streamlit.app/>

Tools Used

- **Transformers (Hugging Face):** The primary library used to download, load, and run the BLOOM-560M or mGPT models.
- **Accelerate:** Used for "Big Model Inference," allowing us to split the model between RAM and Disk to prevent memory crashes.
- **BitsAndBytes:** Provided the 8-bit and 4-bit quantization technology needed to compress large models for low-RAM devices.
- **Generative Models**
 - **BLOOM-560M:** A lightweight, multilingual model (560 million parameters) that supports **Indic languages** (Hindi, Bengali, etc.) while using very little RAM.
 - **mGPT (1.3B):** A larger, more logical multilingual model used when higher story quality was needed.
- **Web & Deployment**
 - **Streamlit:** The web framework used to create the user interface (sliders, text areas, and buttons) using only Python.
 - **GitHub:** Used as the version control and bridge for deploying code to Streamlit Cloud.

Issues Faced with Solution

During the development of the **AI Dungeon Story Generator**, we encountered several technical hurdles. Below is the list of problems faced and the specific solutions implemented to resolve them:

1. Memory Exhaustion (RAM Crash)

- **Problem:** Large multilingual models like **mGPT (1.3B)** require ~5GB–8GB of RAM, exceeding the **1GB limit** on Streamlit Cloud and causing "Out of Memory" (OOM) errors.
- **Solution:** Switched to the **BLOOM-560M** model, which is 10x smaller and fits within **1.2GB of RAM**. For larger models, we implemented **8-bit quantization** using bitsandbytes to compress the model size by 4x.

2. Repetitive "Looping" Text

- **Problem:** The AI often got stuck repeating the same sentence indefinitely (e.g., "The dragon slept. The dragon slept...").
- **Solution:** Integrated a **Repetition Penalty (1.2)** and set `no_repeat_ngram_size=3`. This mathematically penalizes the model for reusing words and strictly forbids repeating any 3-word sequence.

3. Meta-Tensor & Device Mapping Errors

- **Problem:** Using `device_map="auto"` on CPU-only environments triggered a `NotImplementedError` regarding "Meta Tensors" because the library tried to move empty "blueprint" data to the RAM.
- **Solution:** Explicitly set `device=-1` in the pipeline or used the `llm_int8_enable_fp32_cpu_offload=True` flag to force the model to load directly into the system memory without complex GPU mapping.

4. Illogical or "Nonsense" Stories

- **Problem:** At high temperatures, the model picked random, irrelevant words, making the story lose its plot.
- **Solution:** Implemented **Top-P (Nucleus) Sampling (0.9)** and **Top-K (50)**. This filters out the "long tail" of low-probability words, ensuring the AI only chooses from the most logical next steps while maintaining creativity.

5. Multilingual Character Support

- **Problem:** Standard GPT-2 failed to generate or translate Indic scripts (Hindi, Bengali, etc.), resulting in "broken" characters or English responses.
- **Solution:** Migrated to **mGPT** and **BLOOM**, which were natively trained on the **Aksharantar** dataset. We used **Few-Shot Prompting** (providing 1-2 translation examples) to "prime" the model's brain to respond in the target script.

6. Illogical and incorrect translation

- BLOOM is only for the story and we can use **NLLB-200** (No Language Left Behind) for the translation.

Upcoming Output Fixations

1. Add Plot Twist
2. Add Hero name
3. Add Grammar check feature
4. Add Audio output and many more...